

SAT/SMT Based Differential Cryptanalysis of SHA-2

Difference modelling, Bit Conditions and Objective Minimization

July 4, 2026

- ❶ **The idea:** SMT-modelled search for high-probability DCs.
- ❷ **Reading a DC:** an example characteristic; where the 0/1 pins come from; how operations are modelled.
- ❸ **Two models** (difference vs. value); determinism; where contradictions slip by; what the value model fixes.
- ❹ **2024 (Li-Liu-Wang):** Hamming-weight-only objective + a proof that for modular addition this is *exactly* the condition count.
- ❺ **2025 (Li et al., CRYPTO):** Hamming weight + 2-bit conditions; but an over-counting objective.
- ❻ **2026 (Zhang-Li-Gao, main focus):** automated local-collision search + a corrected, complete objective; pipeline + our reproduction.
- ❼ **Message modification:** single-block (SFS) → two-block collision.
- ❽ **Further work.**

The idea: SMT-modelled search for high-probability DCs

- Encode each step operation (modular add, Σ/σ , IF, MAJ, expansion) as constraints on **signed differences**; hand to an SMT solver (STP + CryptoMiniSat).
- A satisfying assignment = a globally-consistent candidate DC.
- **Objective: minimize the number of bit conditions** (equivalently, maximize the DC's probability); work from 2024 minimizes Hamming weight; later work minimizes conditions directly.

Signed difference $\nabla\kappa = (\kappa_v, \kappa_d)$

$(0, 0) = \text{free}$ $(0, 1) = \text{n}$ $(0 \rightarrow 1)$ $(1, 1) = \text{u}$ $(1 \rightarrow 0)$

Bit condition

A constraint that **halves** the conforming-pair probability. Either occurs as a value-pin ($x = 0$) or an (in)equality between two bits ($a = b$, $a \neq b$).

What a differential characteristic is (example)

A DC is, per step i , the signed difference of each register: $\nabla A_i, \nabla E_i, \nabla W_i$; a per-bit string over $\{=, n, u\}$ plus value-pins $\{0, 1\}$. It shows step-by-step of how a particular difference is injected and propagated, for a collision to occur.

Excerpt; 37-step SHA-256 DC (Zhang et al. 2026, Table 8), steps 4–9

i	∇A_i	∇E_i	∇W_i
4	=====	=0=====11===1=====11=0==01	=====
5	=====	=0====00110===0===1==0=11=1=101	=====
6	==n=====u===n=====n===u=u	1nu=00110un00nu0=0=00=11nu0un0nn	nuu=====un===n=====nu==un=u
7	=====	1n001=nnuu0=01u0n11u01u1uu110nun	=====nuuuuuuuuuuuuu===011=====
8	=====	nnn=10011nu0=0101=0011110=110n00	=====
9	=====	1=1===11111===0=1==1111110111110	====nuu=n=====n

n/u = active (a flipped bit, with sign); = = no difference, value free; 0/1 = value is pinned.

A 0/1 is a bit with *no difference* but a *forced value*.

- These are exactly the **bit conditions the nonlinear boolean gates require** for the chosen differences to propagate; imposed on otherwise difference-free bits.
- *Example (IF)*: the transition $\nabla(x, y, z, w) = [==nn]$ propagates only if the selector $x[i] = 0 \Rightarrow$ a 0 pin on a difference-free bit.
- They can be read directly out of the per-transition condition tables. Their count is the exponent in the probability of differential characteristic.

Method. For each operation: (1) enumerate *all valid* bit-level transitions over (input diffs, auxiliaries, output diff) into a truth table; (2) minimize to CNF (LogicFriday); (3) assert the CNF; the solver then only explores valid *difference* transitions.

Modular addition $z = x \boxplus y$ (per bit i)

Tuple of **signed differences**: $(\nabla x_i, \nabla y_i, \nabla c_i, \nabla z_i, \nabla c_{i+1})$ with a carry-difference ∇c ; 27 valid transitions, e.g.

$(=, =, =, =, =), \quad (n, =, =, n, =), \quad (n, =, =, u, n) \dots$

A valid carry-difference chain $\Leftrightarrow$ a conforming value pair exists for that addition.

- Each row $\rightarrow$ clauses over the input, auxiliary and output signed difference variables. The whole hash function is built by chaining these constraints on the specified variables.
- **Addition gates** have the input carry, output carry differences; aside from the inputs' $\nabla x_i, \nabla y_i$ differences.
- **Boolean gates** additionally carry 3 *monitor value bits* (x, y, z) (auxiliary) in the tuple. In literature, usually referred to as the "full-model".
- The extra variables mentioned are the *auxiliary* variables, which make the local transition *deterministic*.

Difference model (the search)

- Variables: (v, d) per bit + auxiliaries (carry-diffs, value monitors).
- Per-operation propagation tables.
- Objective: sparsity + conditions.
- Produces a *candidate* DC.

Value model (validity)

- Real n -bit words, true K_i , full-adder carries.
- Each register's actual value is pinned to its signed-difference symbols.
- Decides whether a conforming pair *exists*.

Why two models

The difference model is a **relaxation**: each operation asserts “ $\exists$ a value locally realizing the transition.”. However, local satisfiability for every piece $\nrightarrow$ global satisfiability. The gap is exactly the set of **invalid DCs**; the value model closes it.

Deterministic Propagation

If a given set of input signed differences yield a single ∇z such that every conforming value pair yields it, i.e. $\Pr_{\text{conforming}}[\text{output} = \nabla z] = 1$.

- **Deterministic:** NOT, shifts, rotations; output difference is fixed by the input difference.
- **XOR / Σ , σ :** activity deterministic, *sign not*: $[n, =, =] \rightarrow$ active, but n vs u depends on the value of the inactive operand (two possible outputs).
- **IF / MAJ:** non-deterministic, e.g. a single ∇ -input can map to two ∇w depending on the hidden data bits.
- **Modular addition:** non-deterministic; one $(\nabla x, \nabla y)$ admits several ∇z via different carry behaviours.

The non-deterministic transitions are made deterministic using auxiliary variables: carry-difference Δc (addition) and monitor values (Boolean gates).

DCs found by difference models are invalid since contradictions are missed. **Root cause:** per-operation existential decoupling; “ $\exists$ a witness for me” $\nRightarrow$ “ $\exists$ one trail-wide value-world.”

- ① **Value monitors untied to arithmetic:** y_t is never forced to equal the value the additions actually compute for E_t .
- ② K_i **invisible** ($\Delta K_i = 0$): only the value model uses it.
- ③ **Modular-difference expansion is value-free:** a fixed modular difference has many signed representations; the solver may pick one no global assignment supports.

These issues are fixed by pinning the signed differences at each step to the actual value bits, and contradictions will be yielded where no value bits can be found. However, the value model is **expensive**, and thus difference is used majorly, and value only for satisfying.

2024 (Li-Liu-Wang): a Hamming-weight-only objective

- First SAT/SMT tool for SHA-2 DCs; two coupled models (difference + value).
- **Objective:** minimize Hamming weight of the signed differences (staged: $\sum H(\nabla W)$, then $\sum H(\nabla A)$, then $\sum H(\nabla E)$).
- **Limitation:** Hamming weight ignores the bit conditions from the *Boolean* functions $\Rightarrow$ the objective is *not* the probability $\Rightarrow$ probabilities re-evaluated by hand.

But Hamming weight is *exactly* right for one part; modular addition

We can prove the number of bit conditions a modular addition imposes equals $H(\nabla z)$ of its output. So the 2024 objective was *optimal for the addition-induced conditions* and only *blind to the Boolean ones*; exactly what 2025/2026 add.

Proof: addition imposes exactly $H(\nabla z)$ conditions (1/2)

E_i is a sum of terms fixed *outside* the addition, so the **modular difference δz is forced** (value-independent). For a valid DC the chosen signed output ∇z is one representation of δz ; its probability is

$$\Pr[\nabla z] = \Pr_z [\pi(z) = \nabla z], \quad \pi(z) = \text{signed diff of } (z, z \boxplus \delta z).$$

With $z' = z \boxplus \delta z$, carry γ ($\gamma_0 = 0$): $z'_i = z_i \oplus \delta_i \oplus \gamma_i$, $\gamma_{i+1} = \text{MAJ}(z_i, \delta_i, \gamma_i)$, using $\text{MAJ}(0, b, c) = b \wedge c$, $\text{MAJ}(1, b, c) = b \vee c$. Per bit:

- **inactive (=)**: need $\delta_i = \gamma_i$, then $\gamma_{i+1} = \delta_i$; if $\delta_i \neq \gamma_i$ no z works. *No value bit fixed.*
- **active (n/u)**: $\delta_i \oplus \gamma_i = 1 \Rightarrow \gamma_i = 1 \oplus \delta_i$ forced; the value bit is forced ($z_i = 1$ for u, 0 for n); $\gamma_{i+1} = z_i$.

The whole carry trail is determined by ∇z alone; independent of the free values.

Proof: addition imposes exactly $H(\nabla z)$ conditions (2/2)

- Fixed value bits = active bits = $H(\nabla z)$; remaining $n - H$ bits free, so $\#\{z\} = 2^{n-H(\nabla z)}$:

$$\Pr[\nabla z] = 2^{-H(\nabla z)} \implies H(\nabla z) \text{ effective bit conditions, and no others.}$$

- These are exactly the self-pins ($n \rightarrow 0, u \rightarrow 1$); nothing else is hidden in the carries.

Why this needs *signed* differences (the crux)

For unsigned differences the count is *not* H : tracking the carry by start/continue/stop relations gives $\#conditions = \#output\ differences$, **minus one** for a contiguous run reaching the MSB (its carry-out is discarded). Signedness is what makes it exactly H .

So the trail-search objective $H(\nabla E_i) + BD(E_i)$ is “addition conditions (H , exact) + Boolean conditions (BD),” disjointly.

2-bit condition: a propagation constraint of the form $a = b$ or $a \neq b$ (two bits, one independent constraint).

Concrete motivation (XOR, in the signed paradigm)

$w = x \oplus y \oplus z$. Take $\nabla x = n$, $\nabla y = \nabla z = =$ (here $x=0$). Then w flips, so ∇w is active; and its *sign* is set by the data: $[n=] \rightarrow n$ iff $y = z$, $\rightarrow u$ iff $y \neq z$. **One 2-bit condition** ($y=z$) either way. But $\nabla x = \nabla y = n$ gives $\nabla w = =$ *deterministically* (no condition). $\Rightarrow$ minimizing Hamming weight does *not* minimize conditions.

- **Model:** keep the full model's monitors (x, y, z) ; append *one* flag bit per transition, $flag = 1 \Leftrightarrow$ a 2-bit condition here.
- **Other Boolean gates:** $\sigma_0, \sigma_1, \Sigma_0, \Sigma_1$ are XOR; the *same* flag table covers them; modular addition, expansion, contradiction detection are untouched.

(x, y, z) -based (joint) counting. The flag counts a condition on the *triple* (x, y, z) of IF/MAJ, and only the *2-bit* subtype.

The flaw

IF at step 17 reads (E_{16}, E_{15}, E_{14}) ; but E_{15}, E_{14} are already **controlled** (free in message modification). Joint counting still charges conditions on them: **non-necessary conditions enter the objective.**

- The minimized number is then *not* the true uncontrolled cost $\Rightarrow$ the optimum is mis-steered; the authors needed **multiple manual attempts** to pin the true minimum on $(E_i)_{15 \leq i \leq 17}$, plus a *manual* probability re-evaluation.
- Also incomplete: only *2-bit* conditions are counted (IF can impose others).

Word-level model (Algorithm 1). For an attack of R steps, a local collision is parameterized by **start step** V and **span** K : differences live only in words $[V, V+K-1]$. Binary ΔW_i (word active?) + a flag f_i per expansion cancellation; a 27-row table encodes the valid $(\Delta W_{i-2}, \Delta W_{i-7}, \Delta W_{i-15}, \Delta W_{i-16}) \rightarrow (\Delta W_i, f_i)$.

$$\text{Objective: } \text{OBJ} = \underbrace{\sum_{i=V}^{V+K} \Delta W_i}_{\# \text{ active words}} + \underbrace{\sum_{i \geq 16} f_i}_{\# \text{ cancellation conditions}} \leq C.$$

- This only chooses **which words carry a difference** (word-level activity). The actual *bit-level* expansion $(\sigma_0, \sigma_1, \text{signs})$ is **not** fixed here; it is handled later, in the trail-search objective.
- **Conditions the paper deems a “good” local collision:** span $14 \leq K \leq 25$; no difference in the first (~ 8) words (for 2-block chaining); $V+K$ small, since conditions after (A_{16}, E_{16}) are hard.
- Go through all parameters (R, V, K) exhaustively ($14 \leq K \leq 25, V \geq 0$); for each, descend the bound C until UNSAT; the last feasible C is the minimum.

Result: 37-step LC = $(W_6, W_7, W_9, W_{14}, W_{22}, W_{23})$, span 18, 6 cancellation conditions; first 6 words clean.

2026: the corrected, complete trail-search objective

Given the local collision, search the DC with the **multi-staged objective**. Older papers (2024 Li, Liu, Wang) also use staged objectives for SHA512 due to the complexity:

- ① O_1 : minimize H of the last few active message words (in the 37-step case W_{22}, W_{23});
- ② $O_2 = \sum_i H(\nabla W_i)$.
- ③ $O_3 = \sum_{i \geq 16} (H(\nabla E_i) + BD(E_i))$; the uncontrolled conditions (the cost / $-\log_2 \text{Pr}$);
- ④ $O_4 = \sum_i H(\nabla A_i)$;
- ⑤ $O_5 = \sum_i H(\nabla E_i)$ (Step 5 also turns on the value-transition model for validity).

(Possibly a weighted combination $\alpha O_1 + O_2$ would likely serve as well.)

Fixing 2025's flaw; two Boolean-model upgrades:

- **Complete conditions** for IF: a pair $(t_1, t_2) \in \{(0, 0), (0, 1), (1, 1)\}$ counts *all* conditions (0/1/2), not only 2-bit.
- **Variable-specific** models (x -based, (x, y) -based, ...): count only on the *uncontrolled* input(s) (step 17: x -based; 18: (x, y) -based; later: full). $\Rightarrow$ objective = exact probability, no manual re-evaluation.

Caveat: do $H(\nabla E)$ and $BD(E)$ over-count in O_3 ?

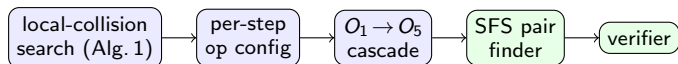
$O_3 = H(\nabla E) + BD(E)$ adds addition conditions and Boolean conditions. Are they truly disjoint? This is unlikely.

- **Case 1: IF Gate** An IF gate can impose either a 1-bit condition or a 2-bit conditions. Condition falls on the selector bit, i.e. in $Ch(x, y, z)$ a y -difference propagates only where $x=1$, so the condition is $x=1$. Thus, overlap if the selector bit is the active bit.
- **Case 2: MAJ Gate** Only places 2-bit conditions, never one-bit conditions, and thus there remains no possibility of double counting.

Possibly Okay

In the *uncontrolled* window, the addition's H term is typically tiny (in the 37-step characteristic, it's 7), and the overlap arises only in a few signed-difference transitions. So any double-counting is usually small for preferred characteristics.

The full pipeline (and our reproduction)



- To check the pipeline: we drove it on the 24-step Sanadhya–Sarkar local collision [10, 11, 12, 13, 17, 18] and produced a **verified** semi-free-start colliding pair (next slide). However, this setup is not ideal for smaller rounds.
- Pinning the per-step trail (a guided solve) found it in ~ 23 s.

Our 24-step SHA-256 result (SS local collision)

Differential characteristic ($O_3 = 1$, i.e. $\text{Pr} = 2^{-1}$; active region steps 10–18):

[illegible]

$O_3 = 1$ here is a single **Boolean (IF) condition**. **Verified semi-free-start colliding pair** (differs only in input words $\{10, 11, 12, 13\}$; words 17, 18 from expansion):

CV_in: b75feccb 293d0b0c d66d41be d8050e1f 8cc675b4 0997b9ac cbeb9f5e 7a55becd

M (W10..W13): 6ff4b62d c0f6e38a 35aa08b5 9d9e9c9a

M' (W10..W13): 6ff4b62c c0f6e38b 35aa68b5 9d9ebc9a

- The objective minimizes the **message**-difference weight; a sparse local collision can still be *state*-dense (high O_3). At reduced rounds, sparsity-ranked collisions give useless O_3 (e.g. 32, 91 for $R = 22, 24$).
- **The method is built for long rounds.** Algorithm 1's "good LC" conditions and the message-weight $\leftrightarrow$ state-cancellation alignment hold in the $R \approx 32\text{--}37$ regime; at short rounds they diverge. *Long-round, high-probability local collisions are where this approach pays off; short rounds need state-aware selection.*

Why it is mandatory. Naively a DC costs $2^{\#\text{conditions}} \gg 2^{128}$; worse than birthday. Message modification satisfies conditions *deterministically* using message freedom.

The control horizon

Solving the state equation for the message word, $W_i = E_i \boxminus A_{i-4} \boxminus E_{i-4} \boxminus \Sigma_1(E_{i-1}) \boxminus \text{IF}(\cdot) \boxminus K_i$, means: *fix the early state $\Rightarrow$ the W_i are determined*. Conditions in steps ≤ 15 are **free**; $W_{\geq 16}$ are expansion-forced $\Rightarrow$ control ends at step 15. Split: controlled (≤ 15) + **uncontrolled** (≥ 16 , count = β).

From SFS to a real collision (two-block). An SFS attack needs a *free* chaining value CV_1 to dial in early state; the true IV gives a fixed one. **Use a first block M_0 to randomize CV_1 :** collide in block 2, vary M_0 until CV_1 is usable. (Mendel'13 / Li'24.)

The non-obvious core

Pre-compute a table of “good” chaining-value prefixes (by solving the dense early part of the trail), indexed by a A_{-1} . Keep resampling M_0 until A_{-1} word matches a table entry; the rest of CV_1 is absorbed *deterministically* by deriving the missing state/message words. The 8 chaining words split as **absorb 7, match 1**.

- Match cost $\sim 2^{32}/|\text{table}|$; everything after the match is forced, then the β uncontrolled conditions are checked.
- **37-step SHA-256**: pre-processing ~ 7 days; $\beta \approx 95$; $T \approx 2^{122.2}$ (match A_{-1}), improved to **$2^{119.1}$** by matching (A_{-2}, A_{-1}) , memory $2^{34.9} - 2^{39.9}$. (36-step: $2^{94.4}$.)

- **Port the 2026 machinery to SHA-512**; the complete/variable-specific Boolean models and the local-collision tool were shown on SHA-256; SHA-512 lags (28 practical / 31 theoretical).
- **Better / longer local collisions**; the explicit 2026 motivation: more steps come first from a better message-expansion local collision.